EduAgentic
AGENTIC AI FOR EDUCATION
MCP · A2A · Jours 33 & 34 · v1.0

JOUR 33 · MINI-PROJET · CORRECTION FINALE

MCP en EdTech : exposer des outils, augmenter un agent

Serveur FastMCP, workflow Langflow, playground et dashboard localhost:8011

AUTEUR	Kamal El Afoui
PROGRAMME	Centre des Très Petites Entreprises Solidaires · DBA Technology
DATE	11 novembre 2026
VERSION	1.0.0 — Correction finale

CE QUE COUVRE CE DOCUMENT

- Serveur MCP FastMCP en transport SSE avec 6 outils EdTech.
- Workflow Langflow prêt à importer et JSON final livré.
- Playground avec trace outil → arguments → latence → résultat.
- Dashboard 8011 : menu, formulaire, graphiques et statistiques.
- Tests, sécurité, limites et mode d'emploi reproductible.

Données 100 % synthétiques · aucun apprenant réel · dépôt reproductible

Résumé exécutif

Un modèle de langage seul ne connaît ni le catalogue de cours, ni le profil d'un apprenant, ni les résultats d'un quiz. Pour agir utilement dans une plateforme EdTech, il doit appeler des outils. Le Model Context Protocol (MCP) standardise cette connexion : le serveur publie une liste d'outils, l'agent les découvre et choisit celui qui répond à la demande.

Ce mini-projet construit EduMCP, un serveur MCP qui expose six outils pédagogiques : recherche de cours, profil apprenant, parcours personnalisé, génération de quiz, correction et statistiques. Le workflow Langflow connecte un agent à ce serveur en SSE ; le playground du dashboard permet d'observer chaque appel. Le tout fonctionne en local, sans donnée réelle, et reste utilisable sans clé API pour les tests déterministes.

Tableau 1 · Synthèse des résultats du mini-projet J33.

Indicateur	Valeur	Preuve
Outils MCP exposés	6 / 6	list_tools + tests REST
Handshake MCP	OK	client officiel Python, transport SSE
Latence médiane des outils	8.2 ms	5 appels mesurés le 11/11/2026
Cours / apprenants	14 / 24	data/courses.json, learners.json
Tentatives de quiz	168	data/quiz_attempts.csv
Stockage	MongoDB Atlas · edtech_*	repli JSON local
Score moyen	74.4/100	calcul sur 168 tentatives
Délégations A2A (J34)	84	data/a2a_delegations.csv

Résultat principal

L'agent n'a pas besoin de connaître l'implémentation du serveur : il découvre les outils, lit leurs schémas JSON et reçoit des résultats structurés. C'est exactement le contrat que MCP standardise.

Ce que démontre le mini-projet

- Un serveur qui publie six outils et leurs schémas, sans que le client ait à coder chaque route.
- Un agent qui sélectionne l'outil adapté et reçoit un résultat JSON validé.
- **Un playground qui rend l'appel observable :** outil, arguments, latence, résultat.
- Un dashboard qui relie la démonstration MCP aux statistiques et au jour 34 (A2A).

Sommaire

Résumé exécutif 2
Ce que démontre le mini-projet 2
1. Contexte et problématique 4
1.1 Pourquoi un standard pour les outils ? 4
1.2 MCP vs REST 4
2. Objectifs et livrables 5
2.1 Objectif de l'atelier 5
2.2 Livrables 5
3. Architecture 6
3.1 Les quatre composants 6
3.2 Cycle d'un appel 6
4. Le serveur MCP EduMCP 7
4.1 Vue d'ensemble 7
4.2 Les six outils 7
4.3 Extrait d'un schéma d'outil 7
4.4 Endpoints du pont REST 8
5. Workflow Langflow 9
5.1 Les quatre nœuds 9
5.2 Import et configuration 9
5.3 Prompt système de l'agent 9
6. Playground et scénarios 11
6.1 Trois scénarios de démonstration 11
6.2 Ce que montre la trace 11
7. Données et jeu de test 12
7.1 KPI calculés 12
8. Tests et validation 13
8.1 Protocole de test 13
8.2 Limites 13
9. Sécurité et bonnes pratiques 14
10. Mode d'emploi 15
10.1 Prérequis 15
10.2 Commandes 15
Conclusion 16
Annexes 17
A. Glossaire 17
B. Checklist de recette 17

1. Contexte et problématique

1.1 Pourquoi un standard pour les outils ?

Dans une plateforme EdTech, un agent peut avoir besoin de chercher un cours, de lire un profil, de recommander un parcours ou de corriger un quiz. Sans standard, chaque intégration ajoute une API spécifique, un format de données différent et une authentification propre. La maintenance devient rapidement plus coûteuse que la valeur apportée.

- **API REST personnalisées :** chaque outil a son endpoint, son format et ses règles.
- **Connecteurs spécifiques :** le code de l'agent dépend de chaque fournisseur.
- **Formats de données différents :** JSON, CSV, texte libre, erreurs non homogènes.
- **Authentification variable :** clé API, OAuth, token interne, aucune unification.

MCP propose un langage commun : le serveur décrit ses outils et leurs paramètres, le client les découvre et les appelle de manière standard. L'agent reste concentré sur la décision, pas sur la plomberie.

1.2 MCP vs REST

Tableau 2 · Deux approches complémentaires : REST pour l'API interne, MCP pour l'agent.

Critère	REST classique	MCP
Contrat	Endpoints propres à chaque API	Interface standard, schémas d'outils
Découverte	Le client connaît les routes à l'avance	Découverte automatique de la liste d'outils
Intégration	Développement spécifique par outil	Réutilisation immédiate par tout client MCP
Évolution	Versionnage manuel des routes	Le serveur publie ses outils et leurs schémas
Dans ce projet	Pont REST du dashboard	Transport SSE + MCP Tools dans Langflow

À retenir

MCP ne remplace pas REST : il décrit comment un agent découvre et utilise des outils. Le dashboard continue d'utiliser un pont REST local, tandis que l'agent Langflow passe par MCP.

2. Objectifs et livrables

2.1 Objectif de l'atelier

Construire un serveur MCP, le connecter à un agent et rejoindre l'écosystème ouvert. Le mini-projet doit être démontrable de bout en bout : un serveur qui tourne, un workflow qui découvre les outils, un playground qui montre les appels et un dashboard qui restitue les résultats.

2.2 Livrables

Tableau 3 · Livrables du jour 33.

Livrable	Fichier	Contenu
Serveur MCP	mcp_server/edumcp_server.py	FastMCP, 6 outils, SSE, pont REST
Workflow Langflow	json/Agentic_AI_EdTech_J33_MCP_Final.json	Chat Input → Agent → MCP Tools → Chat Output
Documentation	docs/Documentation_J33_MCP_EdTech.docx / .pdf	Ce document
Présentation	presentation/J33_MCP_Presentation.pptx	Deck de soutenance
Vidéo	videos/J33_MCP_Workflow_Playground.mp4	Workflow + playground
Dashboard	dashboard/app.py	Streamlit, port 8011

3. Architecture

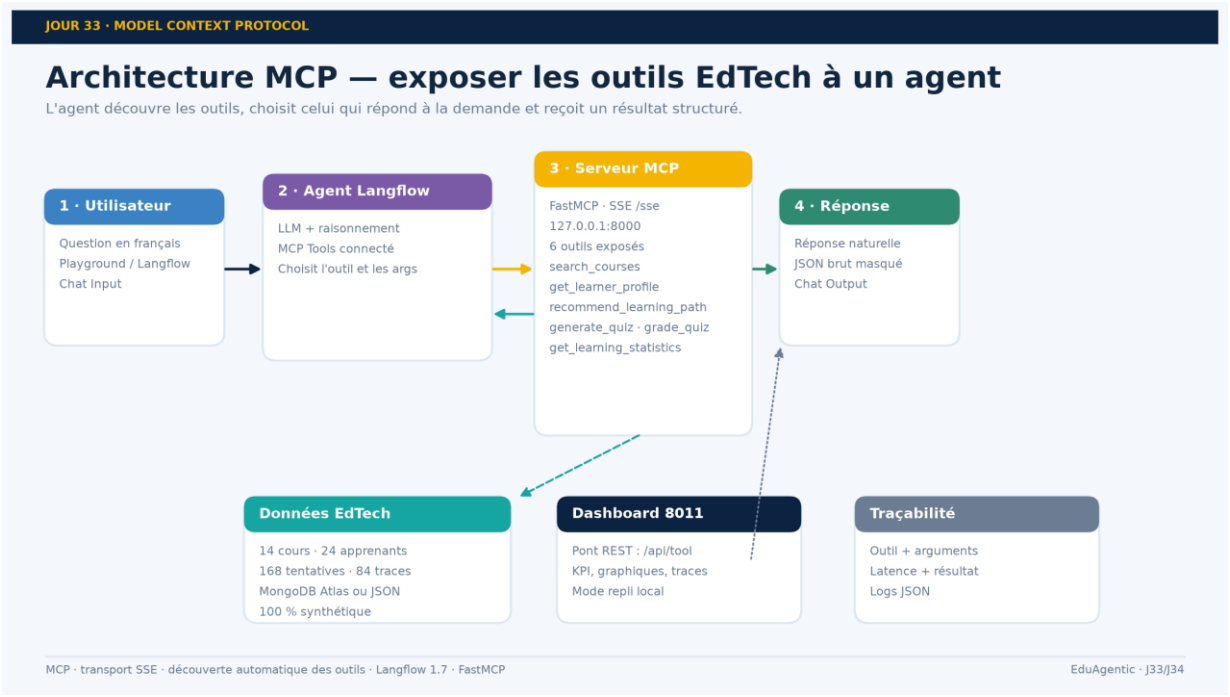


Figure 1 · Architecture MCP : l'agent découvre les outils, le serveur exécute et retourne un résultat structuré.

3.1 Les quatre composants

Tableau 4 · Composants de l'architecture MCP.

Composant	Rôle	Technologie	Port
Playground / Langflow	Envoie la demande, affiche la réponse et la trace	Streamlit / Langflow	8011
Agent	Choisit l'outil et les arguments selon la demande	LLM + MCP Tools	—
Serveur MCP	Expose et exécute les 6 outils	FastMCP + SSE	8000
Données EdTech	Catalogue, profils, quiz, statistiques	JSON / CSV synthétiques	—

3.2 Cycle d'un appel

- **Découverte :** l'agent se connecte au serveur et appelle list_tools.
- **Sélection :** le LLM choisit l'outil adapté à la question et construit les arguments.
- **Exécution :** le serveur valide les paramètres, exécute la fonction et sérialise le résultat.
- **Synthèse :** l'agent reformule le résultat en français, sans exposer le JSON brut.
- **Trace :** le playground enregistre l'outil, les arguments, la latence et le résultat.

4. Le serveur MCP EduMCP

4.1 Vue d'ensemble

Le serveur est construit avec FastMCP. Il tourne sur 127.0.0.1:8000 en transport SSE, avec un point d'entrée /sse et un canal de messages /messages/. Un pont REST local expose /api/health, /api/tools et /api/tool pour le dashboard. Les deux chemins utilisent exactement les mêmes fonctions métier.

Le backend de données est optionnel : si MONGODB_URI est défini dans .env, les collections edtech_courses, edtech_learners, edtech_quiz_bank, edtech_quiz_attempts et edtech_a2a_delegations sont lues depuis MongoDB Atlas. Sinon, le serveur bascule automatiquement sur les fichiers JSON/CSV. Les collections existantes de la base ne sont jamais modifiées.

```python
from mcp.server.fastmcp import FastMCP
from shared import edtech_core as core

mcp = FastMCP(
    "edumcp-edtech",
    instructions="Serveur MCP EduAgentic pour une plateforme d'apprentissage.",
    host="127.0.0.1", port=8000,
    sse_path="/sse", message_path="/messages/",
)

@mcp.tool()
def search_courses(query: str = "", level: str = "", max_results: int = 5) -> dict:
    """Recherche des cours EdTech par mots-clés et filtre par niveau."""
    return core.search_courses(query=query, level=level or None, max_results=max_results)

if __name__ == "__main__":
    mcp.run(transport="sse")
```

4.2 Les six outils

Tableau 5 · Outils exposés par le serveur MCP.

Outil	Paramètres	Résultat	Usage
search_courses	query, level, max_results	Liste de cours + pertinence	Recherche dans le catalogue
get_learner_profile	learner_id	Profil, progression, prochain cours	Personnalisation
recommend_learning_path	learner_id, target_skill	Parcours ordonné + prérequis	Planification
generate_quiz	course_id, difficulty, n_questions	Questions sans réponses	Évaluation formative
grade_quiz	learner_id, course_id, answers	Score, corrections, feedback	Remédiation
get_learning_statistics	segment	KPI + séries pour graphiques	Pilotage

Chaque outil possède une docstring : FastMCP en déduit un schéma JSON (nom, description, types, paramètres requis) que le client peut lire avant l'appel. C'est ce schéma qui permet à l'agent de choisir le bon outil sans connaître le code du serveur.

4.3 Extrait d'un schéma d'outil

```json
```

```
{
  "name": "recommend_learning_path",
  "description": "Parcours ordonné vers une compétence cible.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "learner_id": {
        "type": "string"
      },
      "target_skill": {
        "type": "string"
      }
    }
  }
}
```

4.4 Endpoints du pont REST

Tableau 6 · Points d'entrée du serveur.

Méthode	Route	Usage
GET	/api/health	État du serveur, transport, uptime, liste des outils
GET	/api/tools	Liste lisible des outils et descriptions
POST	/api/tool	Appelle un outil : {tool, arguments}
GET	/sse	Flux SSE du protocole MCP

4.5 Collections MongoDB (optionnel)

Tableau 6b · Collections préfixées edtech_ dans la base fusion.

Collection	Contenu	Volume
edtech_courses	Catalogue des 14 cours	14 documents
edtech_learners	Profils synthétiques	24 documents
edtech_quiz_bank	Questions et corrections	30 documents
edtech_quiz_attempts	Tentatives de quiz	168 documents
edtech_a2a_delegations	Traces A2A	84 documents

5. Workflow Langflow

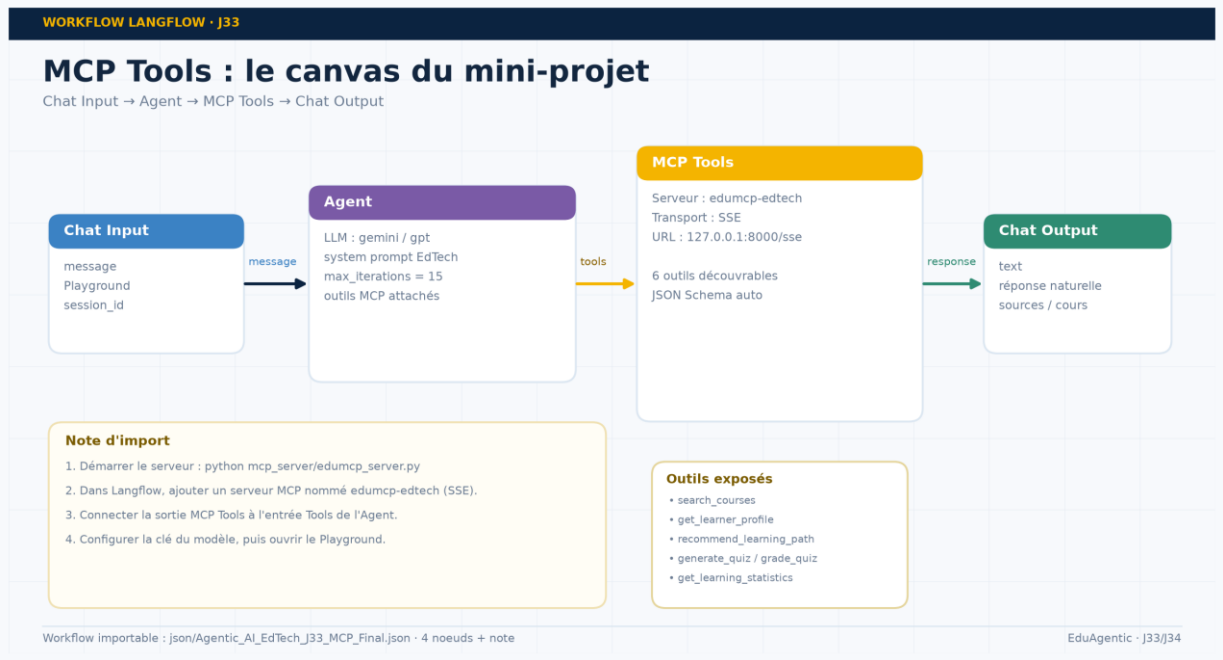


Figure 2 · Canvas Langflow : Chat Input → Agent → MCP Tools → Chat Output.

5.1 Les quatre nœuds

Tableau 7 · Configuration des nœuds Langflow.

Nœud	Configuration	Rôle
Chat Input	message, session_id	Reçoit la question de l'utilisateur
Agent	LLM, system prompt, max_iterations=15	Analyse, choisit l'outil, formule la réponse
MCP Tools	Serveur edumcp-edtech, SSE, 127.0.0.1:8000/sse	Découvre et appelle les outils
Chat Output	text	Affiche la réponse finale

5.2 Import et configuration

- **Lancer le serveur :** python mcp_server/edumcp_server.py.
- Dans Langflow, ajouter un serveur MCP nommé edumcp-edtech, transport SSE, URL http://127.0.0.1:8000/sse.
- Importer json/Agentic_AI_EdTech_J33_MCP_Final.json.
- Vérifier que la sortie MCP Tools est connectée à l'entrée Tools de l'Agent.
- Configurer la clé du modèle (Google Generative AI, OpenAI ou OpenRouter) puis ouvrir le Playground.

Si vous utilisez ngrok

Le serveur MCP peut être exposé par un tunnel HTTPS : ngrok http 8000, puis utiliser l'URL https://<tunnel>/sse dans Langflow. Le tunnel doit rester actif pendant les tests.

5.3 Prompt système de l'agent

```
TEXT
```

```
Tu es un conseiller pédagogique de la plateforme EduAgentic.
Tu disposes d'outils MCP pour chercher des cours, lire un profil, construire un parcours,
générer un quiz, corriger des réponses et calculer des statistiques.

Règles :
- Utilise toujours un outil avant d'inventer une réponse.
- Ne montre jamais le JSON brut : reformule en français, avec des listes claires.
- Cite les identifiants de cours (PY101, DATA350...) et les prérequis.
- Si l'information manque, dis-le et propose une prochaine étape.
- Rappelle que les données sont synthétiques et que la décision finale reste humaine.
```

6. Playground et scénarios

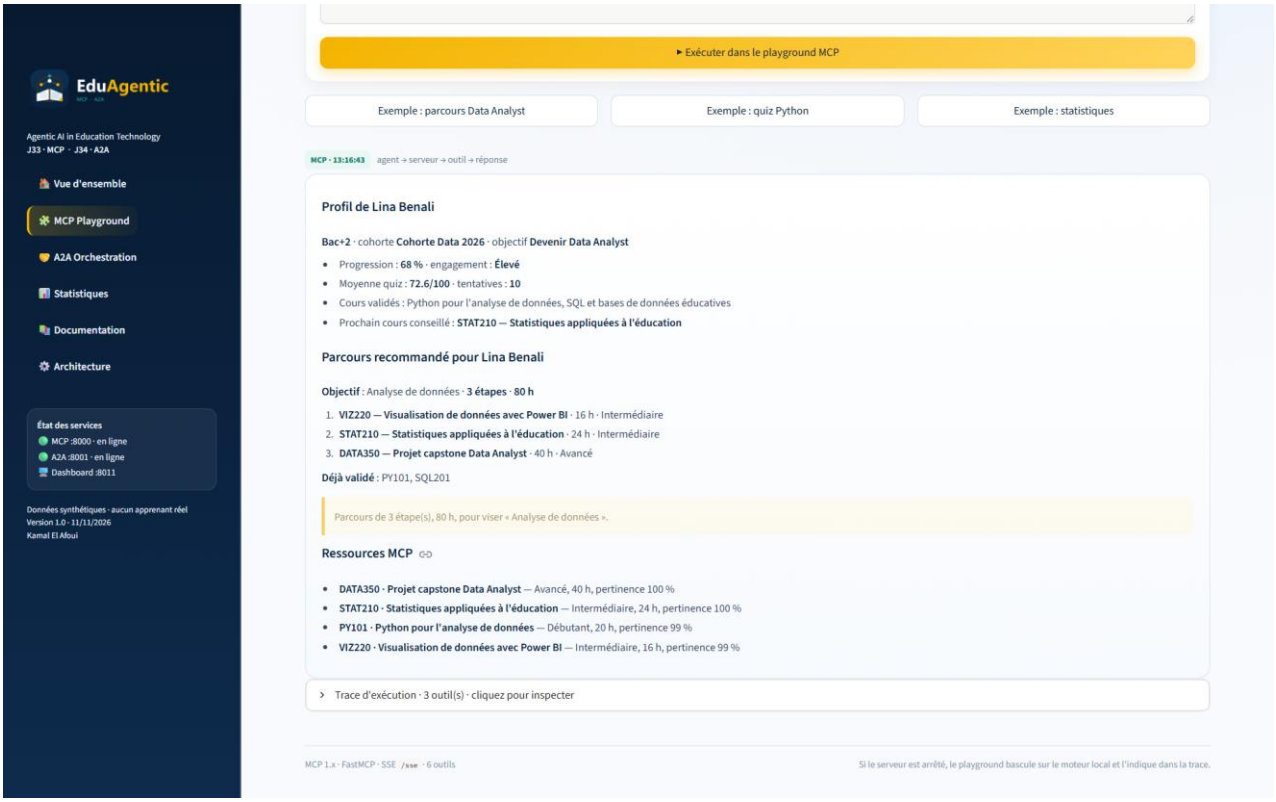


Figure 3 · Playground MCP : profil, parcours recommandé et ressources, avec la trace des outils.

6.1 Trois scénarios de démonstration

Tableau 8 · Scénarios testés dans le playground.

Question	Outils appelés	Résultat attendu
Construis le parcours de Lina vers Data Analyst	get_learner_profile → recommend_learning_path → search_courses	Parcours de 3 étapes, 80 h, avec VIZ220, STAT210 et DATA350
Génère un quiz Python de niveau facile	generate_quiz	3 questions, sans révéler les réponses
Montre-moi les statistiques d'apprentissage	get_learning_statistics	KPI : 24 apprenants, score moyen 74,4, réussite 64,3 %

6.2 Ce que montre la trace

- **L'outil choisi :** search_courses, recommend_learning_path, generate_quiz, etc.
- Les arguments exacts envoyés au serveur, en JSON.
- La latence de l'appel et la source (MCP HTTP ou repli local).
- Le résultat complet, inspectable dans un bloc JSON.
- La réponse reformulée pour l'utilisateur, sans JSON brut.

Mode autonome

Si le serveur MCP est arrêté, le playground bascule sur les mêmes fonctions en local et l'indique dans la trace. La démonstration reste donc possible hors ligne, mais le transport MCP n'est plus exercé.

7. Données et jeu de test

Tableau 9 · Données utilisées par les outils et le dashboard.

Jeu de données	Volume	Fichier
Cours	14 cours, 6 domaines	data/courses.json
Apprenants	24 profils synthétiques	data/learners.json
Banque de questions	30 questions, 6 cours	data/quiz_bank.json
Tentatives de quiz	168 tentatives	data/quiz_attempts.csv
Traces A2A	84 délégations	data/a2a_delegations.csv

Toutes les données sont fictives et déterministes (graine 33). Les prénoms, cohortes et scores sont inventés. Aucun courriel, identifiant réel ou donnée personnelle n'est présent. Le jeu de données peut être régénéré avec python scripts/build_data.py.

7.1 KPI calculés

Tableau 10 · KPI du dashboard, recalculés depuis les données brutes.

KPI	Valeur	Méthode
Apprenants	24	nombre de profils
Cours	14	catalogue
Score moyen	74.4/100	moyenne des 168 tentatives
Réussite quiz	64.3 %	score ≥ 70
Complétion moyenne	71.9 %	moyenne des cours
Inscriptions	14,240	somme des inscriptions du catalogue

8. Tests et validation

8.1 Protocole de test

Tableau 11 · Tests réalisés le 11 novembre 2026.

Test	Commande / action	Résultat
Santé du serveur	GET /api/health	status=ok, 6 outils
Découverte MCP	session.list_tools() via client SSE	6 outils listés
Appel search_courses	call_tool('search_courses', {...})	3 cours retournés
Appel recommend_learning_path	L001 → data-analysis	3 étapes, 80 h
Appel generate_quiz	PY101, 3 questions	QUIZ-..., 3 questions
Appel get_learning_statistics	segment=Tous	KPI et séries
Dashboard	http://localhost:8011	6 pages, 0 exception

Tableau 12 · Mesures des appels d'outils.

Outil	Statut	Latence mesurée	Source
search_courses	OK	11.5 ms	REST
get_learner_profile	OK	8.2 ms	REST
recommend_learning_path	OK	8.1 ms	REST
generate_quiz	OK	6.5 ms	REST
get_learning_statistics	OK	16.6 ms	REST

Interprétation

5/5 appels testés ont répondu sans erreur, avec une latence médiane de 8.2 ms. Les valeurs faibles correspondent au pont REST local ; en production, la latence dépend du réseau, du modèle et du serveur MCP distant.

8.2 Limites

- Le playground utilise un routeur déterministe pour la démonstration ; le workflow Langflow, lui, appelle un LLM.
- Les réponses du LLM ne sont pas évaluées automatiquement dans ce mini-projet : les tests portent sur les outils et le protocole.
- **Le serveur écoute en localhost, sans authentification :** il ne doit pas être exposé tel quel sur Internet.
- Le catalogue est volontairement petit (14 cours) pour rester lisible et reproductible.

9. Sécurité et bonnes pratiques

Tableau 13 · Analyse des risques et mesures.

Risque	Mesure dans le projet	À renforcer en production
Exposition du serveur	Bind 127.0.0.1, tunnel explicite	Authentification, TLS, liste d'IP
Données personnelles	Données 100 % synthétiques	Anonymisation, consentement, RGPD
Injection de prompt	Outils bornés, pas d'exécution arbitraire	Filtrage, allowlist d'outils
Coût des appels	Outils déterministes, pas de LLM dans les tests	Quotas, cache, timeout
Traçabilité	Logs outil + arguments + latence	Journal centralisé, rétention
Erreur d'outil	Messages d'erreur explicites, repli local	Retry, circuit breaker

Principe de moindre privilège

Le serveur ne fait que lire un catalogue synthétique et calculer des scores. Il n'écrit pas dans une base réelle, n'envoie pas de courriel et ne déclenche aucune action irréversible. C'est un choix de conception pour une démonstration sûre.

10. Mode d'emploi

10.1 Prérequis

- Python 3.11 ou supérieur.
- **Dépendances :** pip install -r requirements.txt.
- **Pour le workflow :** Langflow 1.7.0 et une clé de modèle.
- **Pour le dashboard :** Streamlit (inclus dans requirements.txt).

10.2 Commandes

```bash
# 1. Configuration et dépendances
cp .env.example .env          # renseigner MONGODB_URI (ou laisser vide pour JSON)
python scripts/build_data.py
pip install -r requirements.txt

# 2. Serveur MCP (terminal 1)
python mcp_server/edumcp_server.py

# 3. Dashboard (terminal 2)
streamlit run dashboard/app.py --server.port 8011

# 4. Vérification rapide
curl http://127.0.0.1:8000/api/health
curl http://127.0.0.1:8000/sse   # flux SSE (garder ouvert)
```

Tableau 14 · Dépannage rapide.

Problème	Cause probable	Solution
Connexion refusée sur 8000	Serveur non lancé	Démarrer edumcp_server.py
MCP Tools vide dans Langflow	Serveur MCP non déclaré	Ajouter edumcp-edtech en SSE
Réponse lente	Modèle ou tunnel	Tester /api/health, réduire max_iterations
Dashboard sans données	Fichiers data manquants	Relancer scripts/build_data.py

Conclusion

Le mini-projet J33 démontre qu'un serveur MCP peut exposer des outils EdTech de façon standard, lisible et testable. L'agent ne dépend plus des routes internes : il découvre six outils, choisit le bon et reçoit un résultat structuré. Le playground et le dashboard rendent ce fonctionnement observable, ce qui est indispensable pour expliquer et auditer un système agentique.

La suite naturelle est le jour 34 : plusieurs agents spécialisés collaborent via A2A. MCP standardise l'accès aux outils ; A2A standardise la collaboration entre agents. Les deux protocoles se complètent dans une même plateforme d'apprentissage.

Checklist de livraison

☑ Serveur MCP opérationnel · ☑ 6 outils testés · ☑ Workflow JSON final · ☑ Playground · ☑ Dashboard 8011 · ☑ Documentation · ☑ Présentation · ☑ Vidéo.

Annexes

A. Glossaire

Tableau 15 · Glossaire MCP.

Terme	Définition
MCP	Model Context Protocol : standard de découverte et d'appel d'outils par un agent.
FastMCP	Implémentation Python du serveur MCP, avec décorateurs @mcp.tool().
SSE	Server-Sent Events : transport HTTP unidirectionnel utilisé par le serveur.
Outil (tool)	Fonction exposée avec un nom, une description et un schéma de paramètres.
Langflow	Environnement visuel de construction de workflows d'agents.
Playground	Interface de test qui envoie un message et affiche la réponse et la trace.

B. Checklist de recette

- Le serveur démarre et /api/health répond status=ok.
- Le client MCP liste exactement 6 outils.
- Chaque outil retourne un JSON valide sur un cas nominal.
- Le workflow Langflow est importable et les nœuds sont connectés.
- Le playground affiche la trace de chaque appel.
- Le dashboard répond sur localhost:8011 et ne lève aucune exception.